

Module 8 Assignment: Functions and Stored Procedures
CPT 166 – Fundamentals of SQL
Sylvia Schlotterbeck 11/23/25

1. Create Roles

Create three roles:

read_only → can only SELECT from tables.

data_entry → can INSERT into customers and orders but cannot delete data.

admin_assistant → can SELECT, INSERT, UPDATE but cannot create/drop tables:

```
CREATE ROLE read_only;
```

The screenshot shows a SQL execution interface. At the top, a line of SQL code is displayed: `1 CREATE ROLE read_only;`. Below this, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is currently selected and underlined. Under the 'Messages' tab, the text 'CREATE ROLE' is visible. At the bottom of the interface, a status message reads: 'Query returned successfully in 43 msec.'

```
CREATE ROLE data_entry;
```

The screenshot shows a SQL execution interface. At the top, a line of SQL code is displayed: `1 CREATE ROLE data_entry;`. Below this, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is currently selected and underlined. Under the 'Messages' tab, the text 'CREATE ROLE' is visible. At the bottom of the interface, a status message reads: 'Query returned successfully in 42 msec.'

```
CREATE ROLE admin_assistant;
```

The screenshot shows a SQL execution interface. At the top, a line of SQL code is displayed: `1 CREATE ROLE admin_assistant;`. Below this, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is currently selected and underlined. Under the 'Messages' tab, the text 'CREATE ROLE' is visible. At the bottom of the interface, a status message reads: 'Query returned successfully in 49 msec.'

2. Create Users and Assign Roles

Create at least three users:

alice → assign to read_only.

bob → assign to data_entry.

carol → assign to admin_assistant:

```
CREATE ROLE alice LOGIN;  
GRANT read_only TO alice;
```

1

CREATE ROLE alice LOGIN;

2

GRANT read_only TO alice;

Data Output

Messages

Notifications

GRANT ROLE

Query returned successfully in 39 msec.

```
CREATE ROLE bob LOGIN;  
GRANT data_entry TO bob;
```

1

CREATE ROLE bob LOGIN;

2

GRANT data_entry TO bob;

Data Output

Messages

Notifications

GRANT ROLE

Query returned successfully in 41 msec.

```
CREATE ROLE carol LOGIN;  
GRANT admin_assistant TO carol;
```

1

CREATE ROLE carol LOGIN;

2

GRANT admin_assistant TO carol;

Data Output

Messages

Notifications

GRANT ROLE

Query returned successfully in 41 msec.

3. Grant and Revoke Privileges

Grant the roles the correct privileges on the appropriate tables.

Demonstrate revoking a privilege (e.g., remove UPDATE from admin_assistant on products):

```
GRANT SELECT ON customers, orders, order_details, products, categories TO read_only;
```

```
1 GRANT SELECT ON customers, orders, order_details, products, categories TO read_only;
```

Data Output Messages Notifications

GRANT

Query returned successfully in 40 msec.

```
GRANT INSERT ON customers, orders TO data_entry;
```

```
1 GRANT INSERT ON customers, orders TO data_entry;
```

Data Output Messages Notifications

GRANT

Query returned successfully in 104 msec.

```
GRANT SELECT, INSERT, UPDATE ON customers, orders, order_details, products, categories TO admin_assistant;
```

```
1 GRANT SELECT, INSERT, UPDATE ON customers, orders, order_details, products, categories TO admin_assistant;
```

Data Output Messages Notifications

GRANT

Query returned successfully in 64 msec.

```
REVOKE UPDATE ON products FROM admin_assistant;
```

```
1 REVOKE UPDATE ON products FROM admin_assistant;
```

Data Output Messages Notifications

REVOKE

Query returned successfully in 52 msec.

4. Test Permissions

As each user (alice, bob, carol), attempt the following actions:

alice: run a SELECT on customers (should succeed).

bob: attempt to DELETE from orders (should fail).

carol: UPDATE a product's price (should succeed). Then try DROP TABLE products (should fail).

SET ROLE alice;

1	SET ROLE alice;	
Data Output	<u>Messages</u>	Notifications
SET		
Query returned successfully in 52 msec.		

SELECT * FROM customers;

1 SET ROLE alice;

2 SELECT * FROM customers;

Data Output

Messages

Notifications

Showing rows: 1 to 91

Page No: 1

of 1

customer_id [PK] integer

customer_name character varying (255)

contact_name character varying (255)

address character varying (255)

city character varying (255)

postal_code character varying (255)

1 1 Alfreds Futterkiste Maria Anders Obere Str. 57 Berlin 12209

2 2 Ana Trujillo Emparedados y helad... Ana Trujillo Avda. de la Constitucion 2222 Mexico D.F. 05021

3 3 Antonio Moreno Taquera Antonio Moreno Mataderos 2312 Mexico D.F. 05023

4 4 Around the Horn Thomas Hardy 120 Hanover Sq. London WA1 1DP

5 5 Berglunds snabbkoop Christina Berglund Berguvsvegen 8 Lulea S-958 22

6 6 Blauer See Delikatessen Hanna Moos Forsterstr. 57 Mannheim 68306

7 7 Blondel pere et fils Frederique Citeaux 24, place Kleber Strasbourg 67000

8 8 Bolido Comidas preparadas Martin Sommer C/ Araquil, 67 Madrid 28023

9 9 Bon app Laurence Lebihans 12, rue des Bouchers Marseille 13008

10 10 Bottom-Dollar Marketse Elizabeth Lincoln 23 Tsawassen Blvd. Tsawassen T2F 8M4

11 11 Bs Beverages Victoria Ashworth Fauntleroy Circus London EC2 5NT

12 12 Cactus Comidas para llevar Patricio Simpson Cerrito 333 Buenos Aires 1010

13 13 Centro comercial Moctezuma Francisco Chang Sierras de Granada 9993 Mexico D.F. 05022

14 14 Chop-suey Chinese Yang Wang Hauptstr. 29 Bern 3012

15 15 Comercio Mineiro Pedro Afonso Av. dos Lusíadas, 23 Sao Paulo 05432-043

16 16 Consolidated Holdings Elizabeth Brown Berkeley Gardens 12 Brewery London WX1 6LT

17 17 Drachenblut Delikatessend Sven Ottlieb Walserweg 21 Aachen 52066

18 18 Du monde entier Janine Labruno 67, rue des Cinquante Otages Nantes 44000

19 19 Eastern Connection Ann Devon 35 King George London WX3 6FW

20 20 Ernst Handel Roland Mendel Kirchgasse 6 Graz 8010

21 21 Familia Arquibaldo Aria Cruz Rua Oros, 92 Sao Paulo 05442-030

22 22 FISSA Fabrica Inter. Salchichas S.A. Diego Roel C/ Morazarzal, 86 Madrid 28034

23 23 Folies gourmandes Martine Rance 184, chaussee de Tournai Lille 59000

24 24 Folk och fe HB Maria Larsson Akergatan 24 Brecke S-844 67

Total rows: 91

Query complete 00:00:00.072

CRLF

Ln 2, Col 14

SET ROLE bob;

1	SET ROLE bob;	
Data Output	Messages	Notifications
SET		
Query returned successfully in 62 msec.		

DELETE FROM orders
WHERE order_id > 11011;

1	SET ROLE bob;	
2	DELETE FROM orders	
3	WHERE order_id > 11011;	
Data Output	Messages	Notifications
ERROR: permission denied for table orders		
SQL state: 42501		

SET ROLE carol;

1	SET ROLE carol;	
Data Output	Messages	Notifications
SET		
Query returned successfully in 39 msec.		

(I had to first restore update privileges to admin_assistant before Carol could update the products table, since I had revoked them previously):

```
SET ROLE postgres;
GRANT UPDATE ON products TO admin_assistant;
SET ROLE carol;
UPDATE products
    SET price = 36.36
    WHERE price = 33.33
```

```
1  SET ROLE postgres;
2  GRANT UPDATE ON products TO admin_assistant;
3  SET ROLE carol;
4  UPDATE products
5      SET price = 36.36
6      WHERE price = 33.33;
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 47 msec.

DROP TABLE products;

```
1  SET ROLE carol;
2  DROP TABLE products;
```

Data Output Messages Notifications

ERROR: must be owner of table products

SQL state: 42501

Short reflection:

Roles and permissions are important in database security in similar ways that locking down primary and foreign keys are to ensuring the integrity and reliability of data in tables and the relationships between different tables in a schema. Having a deliberate structure of access and control in a database allows users to access exactly what they need in a database, and prevent any extraneous access to each user. This greatly increases the data integrity of the database when a thoughtfully planned-out access structure is created and implemented, allows easier management of the database by the admin. Defining who can and cannot do exactly which actions on exactly which tables is a powerful, granular tool that Postgres offers. If everyone had Superuser privileges, and it would be similar if any kind of random data could be entered into primary and foreign keys—it would throw the entire reliability and usability of the database into a very precarious place. It would be possible for any random user who had access to the database to completely destroy the database with a few errant keystrokes.